

Trabajo Práctico Final – Algoritmos y Estructuras de Datos - Comisión 5

Grupo: Alan leslie, Axel Sanchez, Tomás Guerrero

Introducción:

El objetivo del trabajo fue analizar archivos CSV y determinar cuáles son los elementos que poseen la mayor cantidad de ocurrencias dentro del dataset.

Al iniciar la aplicación, el usuario selecciona un archivo CSV. Luego el sistema carga los datos en una lista de cadenas de texto (`List<string>`) y permite realizar búsquedas utilizando dos estrategias diferentes:

- Búsqueda utilizando una estructura Heap (Max-Heap).
- Búsqueda utilizando un método de ordenamiento alternativo.

Además, el sistema permite ejecutar distintas consultas sobre la Heap generada y comparar tiempos de ejecución entre los algoritmos implementados.

Métodos implementados

BuscarConHeap:

Este método obtiene los elementos con mayor cantidad de ocurrencias utilizando una estructura Max-Heap.

Funcionamiento:

Este método fue implementado en su totalidad. Consta de tres pasos: en primer lugar, cuenta las ocurrencias de cada elemento usando un diccionario; segundo, construye una Max-Heap insertando cada elemento con su frecuencia; tercero, extrae los N elementos con mayor cantidad de ocurrencias y los devuelve en la lista collected. Para el correcto funcionamiento de la Heap se implementaron dos métodos auxiliares privados: FiltrarArriba, que restaura la propiedad de la Heap luego de una inserción, y FiltrarAbajo, que la restaura luego de una extracción del máximo. Este método nos permite obtener rápidamente los elementos más frecuentes sin necesidad de ordenar completamente todos los datos.

BuscarConOtro:

Este método obtiene los elementos más frecuentes utilizando un algoritmo de ordenamiento.

Funcionamiento:

Este método fue implementado utilizando un algoritmo de ordenamiento basado en Selection Sort. Al igual que BuscarConHeap, primero cuenta las ocurrencias, luego ordena la lista de mayor a menor por frecuencia y finalmente toma los primeros N elementos para devolverlos en collected. Es una solución sencilla y fácil de comprender.

Consulta1:

Funcionamiento:

Este método mide y compara los tiempos de ejecución de BuscarConHeap y BuscarConOtro al buscar los 5 elementos con mayor cantidad de ocurrencias. Se utilizó la clase Stopwatch de C# para medir el tiempo en milisegundos de cada método por separado, retornando ambos resultados en un único string.

Consulta2:

Este método obtiene el camino desde la raíz hasta la hoja más izquierda de la Heap.

Funcionamiento:

La consulta construye la misma Max-Heap utilizada en el método de búsqueda y obtiene el camino desde la raíz hasta la hoja más izquierda. Para ello, comienza en el nodo raíz y recorre sucesivamente los hijos izquierdos hasta llegar a una hoja, devolviendo el recorrido en formato texto.

Consulta3:

Funcionamiento:

Este método construye la Max-Heap a partir de los datos de entrada y muestra todos los elementos que la componen, indicando además el nivel en el que se encuentra cada nodo dentro del árbol. Esto permite visualizar la estructura completa de la Heap generada.

Problemas encontrados y soluciones

Problemas en consulta2:

Uno de los problemas fue que existían varios caminos con la misma profundidad, encontramos la solución conservando el primer camino encontrado con profundidad máxima. Esto simplificó la implementación y cumplía con los requisitos del método.

Además, al principio los nodos se mostraban uno al lado del otro, lo que dificultaba su lectura, por eso mismo, utilizamos un separador/flecha para representar visualmente el recorrido desde la raíz hasta la hoja más profunda.

Problemas en consulta3:

La principal problemática en el desarrollo fue el cómo plantear el retorno de los datos solicitados por el enunciado, el cual se resolvió aplicando una lógica matemática que permite mostrar los niveles correspondientes a cada nodo y su respectivo valor con una condicional IF.

Al detectar que se completa un nivel, incrementa el contador del nivel actual y lo muestra 1 vez para indicar el nivel en el que están los datos retornados por pantalla.

Además otro problema que se pudo observar fue que el programa otorgado por la catedra, presenta un error en la lectura del .csv

Este presenta una estructura de datos separada por punto y coma (";") y en la lectura el Split esta definido por (",") lo que genera un problema en la mayoría de ejecuciones.

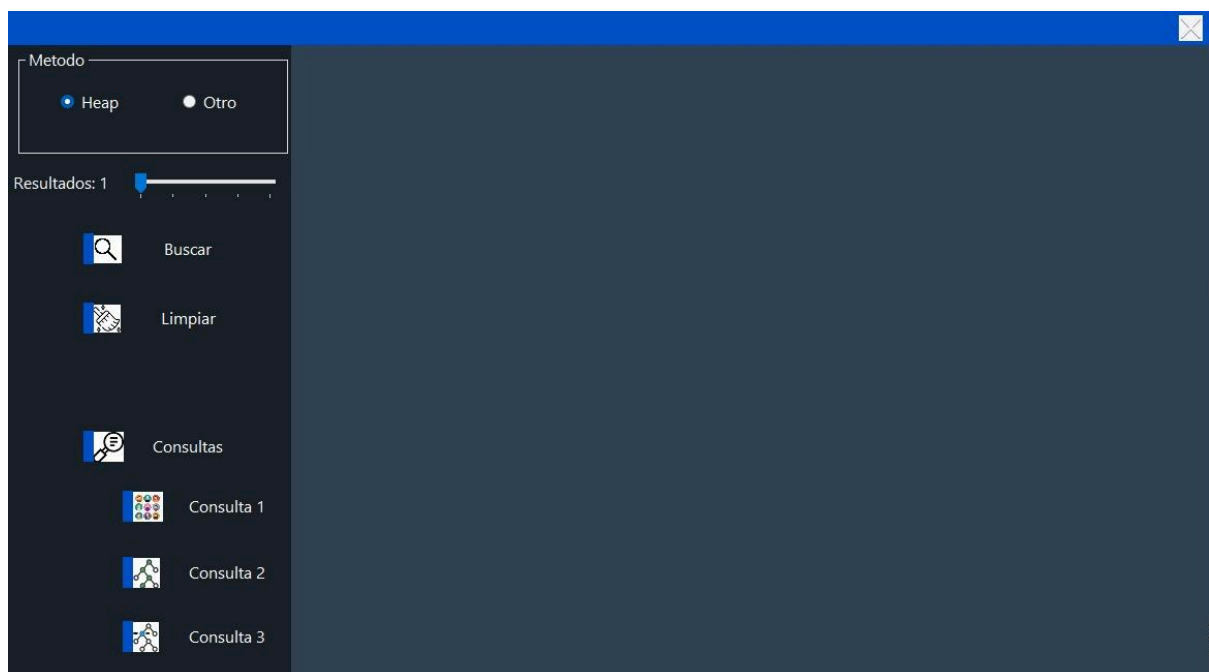
Pantallas del sistema:

Pantalla inicial:



Permite seleccionar el archivo CSV y cargar los datos para su posterior procesamiento.

Pantalla principal:



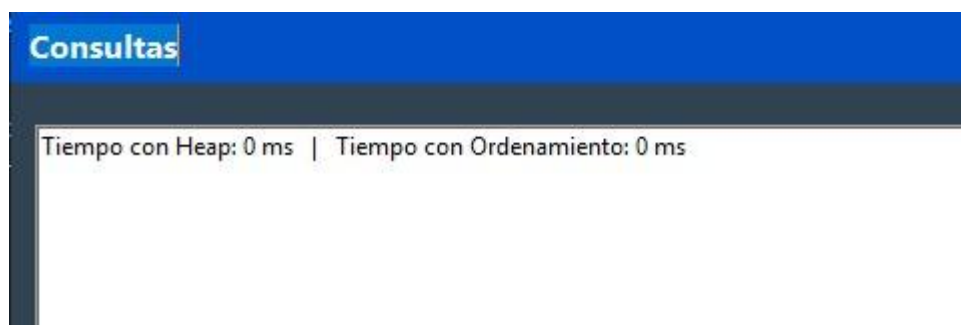
Metodo			
☒ Heap ☐ Otro	✓	Mean-CD	Ocurrencias: 5428
Resultados: 5	✓	Mean-UHF42	Ocurrencias: 3864
Buscar	✓	Mean-UHF34	Ocurrencias: 3127
Limpiar	✓	Estimated annual rate age 18-UHF42	Ocurrencias: 504
Consultas	✓	Estimated annual rate under age 18-UHF42	Ocurrencias: 504
Consulta 1			
Consulta 2			
Consulta 3			

Permite:

- Elegir el método de búsqueda.
- Seleccionar la cantidad de resultados.
- Ejecutar la búsqueda.
- Visualizar los resultados obtenidos.

Pantalla de consultas:

Consulta1:



Consulta2:

Consultas

Camino a la hoja más izquierda de la Heap: (5428) Mean-CD



(3127) Mean-UHF34



(504) Estimated annual rate age 18-UHF42



(354) million miles-CD



(252) Annual average concentration-UHF42

Consulta3:

Consultas

Nivel 0: (5428) Mean-CD
Nivel 1: (3127) Mean-UHF34
Nivel 1: (3864) Mean-UHF42
Nivel 2: (504) Estimated annual rate age 18-UHF42
Nivel 2: (504) Estimated annual rate under age 18-UHF42
Nivel 2: (252) Number per km2-UHF42
Nivel 2: (252) million miles-UHF42
Nivel 3: (354) million miles-CD
Nivel 3: (460) Mean-Borough
Nivel 3: (504) Estimated annual rate-UHF42
Nivel 3: (60) Estimated annual rate age 18-Borough
Nivel 3: (60) Estimated annual rate-Borough
Nivel 3: (60) Estimated annual rate under age 18-Borough
Nivel 3: (30) Number per km2-Borough
Nivel 3: (20) million miles-Borough
Nivel 4: (252) Annual average concentration-UHF42
Nivel 4: (30) Annual average concentration-Borough
Nivel 4: (168) Estimated annual rate age 30-UHF42
Nivel 4: (118) Annual average concentration-CD
Nivel 4: (20) Estimated annual rate age 30-Borough
Nivel 4: (92) Mean-Citywide
Nivel 4: (6) Number per km2-Citywide
Nivel 4: (12) Estimated annual rate-Citywide
Nivel 4: (4) Estimated annual rate age 30-Citywide
Nivel 4: (6) Annual average concentration-Citywide
Nivel 4: (12) Estimated annual rate age 18-Citywide
Nivel 4: (6) million miles-Citywide
Nivel 4: (12) Estimated annual rate under age 18-Citywide

Permite ejecutar las consultas adicionales implementadas sobre la estructura heap.

Posibles mejoras:

Si bien lo que implementamos cumple con los requisitos planteados en el enunciado, existen varias mejoras que podrían incorporarse en futuras versiones. En principio, sería optimizar los algoritmos de búsqueda y ordenamiento utilizados en las consultas. Por ejemplo, en la Consulta 1 podría utilizarse una estructura más eficiente para obtener los elementos más frecuentes sin necesidad de ordenar todos los datos.

También podría ampliarse la funcionalidad de la Consulta 2 permitiendo mostrar no solo el camino más profundo del árbol, sino también todos los caminos de igual profundidad o información adicional sobre cada nodo recorrido. Otra mejora posible sería incorporar nuevas consultas estadísticas sobre el conjunto de datos, brindando al usuario más herramientas para analizar la información almacenada.

Desde el punto de vista de la interfaz gráfica, podría agregarse una visualización del árbol generado, permitiendo observar gráficamente la estructura de nodos y las relaciones entre ellos. Esto facilitaría la comprensión de los resultados obtenidos por las consultas. Finalmente, una versión más avanzada podría permitir trabajar con distintos archivos CSV, guardar resultados de consultas, exportar informes y comparar estadísticas entre diferentes conjuntos de datos.

Conclusión final:

La realización de este trabajo permitió aplicar de manera práctica los conceptos estudiados en la materia, especialmente el uso de estructuras de datos como Heap, algoritmos de ordenamiento y diccionarios. También permitió comprender las diferencias de rendimiento entre distintos algoritmos y reforzar conocimientos relacionados con complejidad, ordenamiento y búsqueda de información. La experiencia fue bastante útil para comprender cómo utilizar estructuras de datos en problemas reales y desarrollar aplicaciones completas en C#.